大语言模型混合量化压缩与加速推理技术

尹经纬，李志强+，刘裕彤

（扬州大学 信息工程学院，江苏 扬州 225009）

摘 要：大语言模型已广泛应用于日常学习、工作和生活中，但由于其参数规模庞大、资源消耗高，且推理高度依赖GPU，这严重制约其推广。针对上述问题，论文在CPU环境下提出基于离群特征优化的混合INT8量化方法，充分发挥其在模型压缩中的优势；同时，基于注意力机制在文本首尾集中分布的规律，设计高效的参数快速读取机制。两种方法的有机结合显著减少模型内存消耗和提升推理效率，为解决大语言模型在边缘计算环境中的应用瓶颈提供新的技术方案。在I7-13700 CPU环境下，基于LLaMA2、GPT-J和FSEQ大模型，使用C4、Wikitext和PG19数据集进行全面验证，结果充分验证了所提方法的优越性与实用价值。

关键词：大语言模型；离群参数；混合量化；注意力机制；参数快速读取；模型推理；边缘计算

中图法分类号：TP183 **文献标识码**：A **文章编号**：1000-7024（2026）01-0187-08

doi：10.16208/j.issn1000-7024.2026.01.023

Hybrid quantization compression and accelerated inference techniques for large language models

YIN Jing-wei，LI Zhi-qiang+，LIU Yu-tong

（School of Information Engineering，Yangzhou University，Yangzhou 225009，China）

Abstract：Large language models（LLMs）have been widely adopted in daily learning，work，and life. However，their enormous parameter sizes，high resource consumption，and reliance on GPUs for inference severely limit their widespread adoption. To address these challenges，a hybrid INT8（HI8）quantization method optimized using outlier parameters in a CPU environment was proposed to leverage its advantages in model compression. Simultaneously，a high-speed parameter rapid（HSR）retrieval mechanism was designed based on the observed pattern that attention mechanisms tend to concentrate at the beginning and end of texts. Memory consumption was substantially reduced，and inference efficiency was enhanced by the combination of these two methods，offering a novel technical solution to overcome the deployment bottlenecks of LLMs in edge computing scenarios. Comprehensive evaluations on the I7-13700 CPU environment using LLaMA2，GPT-J，and FSEQ models，with datasets such as C4，Wikitext，and PG19，demonstrate the superiority and practical value of the proposed approach.

Key words：large language models；outlier parameters；hybrid quantization；attention mechanism；fast parameter access；model inference；edge computing

0 引 言

大语言模型（large language models，LLMs）是通过大规模文本数据训练的深度学习模型，其面临着普遍存在的问题，即大量参数优化需要长时间的训练，而且训练过程高度依赖价格昂贵的GPU，这造成了运行成本居高不下。且对普通消费者来说，无法基于CPU来本地使用大模型，这都严重制约着大模型推广。

针对上述问题，Frantar等[1]提出无需重复训练的一次性剪枝策略以提高运行效率。Wang等[2]提出了一种针对所有参数设置权重的方法，利用小型校准数据集进行近似，从而删除权重低的参数来压缩模型大小。上述方法虽然提

收稿日期：2024-09-26；**修订日期**：2025-12-18

基金项目：国家自然科学基金项目（62071240）；江苏省水利科技基金项目（合同号：2023055）

作者简介：尹经纬（1999-），男，江苏扬州人，硕士研究生，研究方向为人工智能、量子计算；+通讯作者：李志强（1974-），男，江苏泰州人，博士，教授，硕士生导师，研究方向为量子计算、人工智能；刘裕彤（1999-），男，江苏南通人，硕士研究生，研究方向为量子计算、人工智能。E-mail：410892156@qq.com

中国知网 https://www.cnki.net

高训练速度，但仍然无法解决模型训练时间长和能耗高的问题，并且它们无法应用在CPU中来实现高效的推理。

为了进一步解决问题，本文提出混合量化压缩与加速推理技术，拟在CPU环境中通过识别和优化离群参数来实现混合INT8量化（hybrid int8 quantization，HI8）；同时，基于注意力机制在文本首尾集中分布的规律，设计参数快速读取机制（high-speed parameter red，HSR）。有机结合两种方法在内存消耗和推理效率方面取得了显著提升，使大语言模型在CPU环境中进行性高效推理。本文主要贡献如下：

（1）在原float32模型中，通过少量数据集找出离群参数，针对离群参数采用float16运算，其余采用INT8量化，从而可以保留精度的同时大幅度压缩模型；

（2）重复利用定量推理过程参数，有效避免缓存不断增加导致内存溢出问题，因而显著提高模型吞吐量；

（3）将HI8量化和HSR相结合的模型在保持整体性能的同时，内存消耗降低3.5倍，吞吐量提升5倍。

1 相关工作

1.1 模型量化

大语言模型通常拥有数十亿甚至上百亿的参数量，比如GPT-3模型包含1750亿个参数[3]。这使得在模型推理过程中需要大量的内存来存储模型的权重和中间激活值，对计算资源的要求非常高，特别是在部署到边缘设备或移动设备上时，内存限制会成为一个严重的问题[4]。为了减少内存消耗和计算量，常用的一种方法是模型量化。模型量化基本思想是用较少的位数表示模型参数，从而减少内存消耗和计算量，同时保持尽可能高的模型性能。常见的模型量化[5]方法包括二值量化、定点数量化和整数量化。

（1）二值量化：将参数量化为二进制数[6]，通常是+1或−1。这种方法可以极大地减小模型的大小和计算量，但会导致较大的精度损失；

（2）定点数量化：将浮点参数转换为定点数表示[7]。定点数通常为整数或带有固定小数位的数字。例如，将参数乘以一个比例因子，并将结果四舍五入到最接近的整数或固定小数点位数，以实现参数的量化；

（3）整数量化：将参数量化为整数，通常是8位或更少位的整数[8]。整数量化可以通过将浮点数参数乘以一个比例因子，然后将结果四舍五入到最接近的整数来实现。

其中，整数量化是应用最广泛的方法，尤其是INT8量化[9,10]。常规INT8量化存在着显著缺点：

（1）INT8量化方法需要在量化后对模型进行重新训练[11]，以调整模型参数和优化量化后的性能，这会消耗较多的计算资源和时间；

（2）对于参数规模较大的模型（超过10亿），INT8量化的效果可能会受到限制，因为大模型参数中常伴随着离群值，会导致参数量化精度降低，这些误差在模型中层层传递累加后会降低模型性能，甚至会导致模型失效。因此，在应用INT8量化时需要权衡精度和模型性能的需求，选择合适的量化方法和参数设置。

1.2 KV Cache

大模型采用Transformer的架构模式[12]。其结构中的核心部件是self-attention，其计算逻辑为

$$\mathrm{Attention}(Q,K,V)=\mathrm{softmax}(\frac{QK^T}{\sqrt{d_k}})V \tag{1}$$

其中，$Q\in R^{l\times d}$，$K\in R^{l\times d}$，$V\in R^{l\times d}$分别表示token的query、key和value的矩阵[13]，l表示分词token个数，d表示嵌入维度。大模型在推理时候，是一种自回归的推理模式，一般情况下大模型每次在推理下一个token输出时，会在self-attention中计算所有的token之间的相互注意力权重[14]，而每次只会采用最后一个token的输出来预测下一个token，从而造成大量的冗余计算。

为避免大模型在self-attention层进行冗余计算，提升大模型的推理速度，可引入KV Cache[15]。KV Cache是通过缓存self-attention机制中的Key和Value，避免生成每个Token时，对先前的token重复计算。KV Cache的这种以空间换时间的加速技术，显著提高了模型的生成速度。传统KV Cache技术分为初始化，使用缓存和更新3个步骤[16]：

（1）初始化：依据输入分词token集合来获取每个token的Key和Value，更新到KV Cache缓存中备用，并计算预测出下一个token；

（2）使用缓存：假设模型预测执行到第t步，获取所有缓存的Key和Value来计算第t步token的输出；

（3）更新缓存：将上步计算的第t步token的key和value拼接到Key和Value缓存中。

尽管目前KV Cache技术在推理加速方面的效果非常突出，但是其KV缓存的内存消耗会随着大模型参数的增长和序列长度增加而迅速增长[17]，甚至会远超出模型大小，为解决这个问题，目前采用的常用的办法是删除低权重的KV对[18]，压缩序列长度[19]，量化KV参数[20]来压缩缓存的大小。

2 混合量化压缩与加速推理

2.1 HI8量化

为尽可能保障量化后的模型推理精度，最热门的量化策略是向量式量化[21]，它对隐藏状态的每一行进行量化，这样可以有效避免离群特征[22]对量化性能的影响，但在参数规模较大的模型上，向量式量化的效果受到了严重限制，其推理速度和泛化能力都受到了很大影响。

在对参数规模达到亿级以上的模型进行离群特征的深入分析后发现，离群特征在隐含状态中的分布位置具有高

中国知网 https://www.cnki.net

度集中性，且这种规律在模型的所有层中表现出一致性。尽管离群特征数量较少，但其对模型性能的影响非常显著。通过以下3个特性可以精确定位离群特征的位置：①隐含状态的绝对值大于6；②超过60%的层在相同位置存在离群特征；③离群特征在20%的序列维度中出现。基于上述特性，本文对LLaMA2 7B[22]、GPT-J 6B[22]和FSEQ 6.7B[22]这3种大语言模型进行详细分析与验证，具体方法与实验步骤如下，实验结果见表1。

表1　不同模型离群特征的统计结果

LLM	困惑度			个数	维度	层数/%	序列/%	值范围
	F32	S0	R0					
LLaMA2	5.6	11.3	6.0	7	6	64	47	[6,18]
GPT-J	13.8	21.4	14.5	6	6	62	28	[14,21]
FSEQ	13.3	17.9	13.6	6	6	100	75	[35,44]

步骤1　利用少量C4数据集（详见3章节），根据离群特征的判定条件，分别识别出LLaMA2 7B、GPT-J 6B和FSEQ 6.7B模型中离群特征的位置；

步骤2　针对每个模型构建3种对比模型：①原始float32模型，作为基准参考，称为F32；②将离群特征位置的参数置为0的模型，称为S0；③随机选取等比例（离群特征占总隐含状态的比例）位置并置为0的模型，称为R0；

步骤3　使用EleutherAI工具（详见3章节）在C4数据集上对以上模型进行性能评估，困惑度（Perplexity）作为主要衡量指标（详见3.1章节）。

从表1中结果中可以得到如下结论：

（1）离群特征的分布规律，离群特征通常出现在隐含状态中的6或7个具体位置，且这些特征仅涉及6个维度。在LLaMA2 7B和GPT-J 6B中，离群特征分别出现在64%和62%的层同样位置中，而在FSEQ 6.7B中，这一比例高达100%；

（2）离群特征的绝对值范围在不同模型间虽然有所差异性，但其绝对值都是大于6的；

（3）离群特征对模型性能的影响，将离群特征位置置为0（S0模型）导致困惑度显著增加。如，LLaMA2 7B的困惑度从5.6增加到11.3，增幅达1.99倍；GPT-J 6B和FSEQ 6.7B的困惑度分别增加1.55和1.35倍。而R0模型对困惑度的影响明显较小。以LLaMA2 7B为例，困惑度仅从5.68增加到6.04，增幅仅为1.06倍。这表明离群特征对模型整体性能的贡献占比高达59%，对推理准确率具有决定作用。

因此基于离群特征的特性，本文在CPU有限计算资源前提下，提出HI8量化方法，即在原有的float32参数中找出离群特征，针对离群参数采用float16运算，其余的采用INT8量化。具体做法如图1所示。在图1中，我们展示了矩阵乘法的混合精度分解，其中将离群值特征维度O分为两部分进行处理：

（1）处理离群值特征维度 O 的部分

$$Cf32 \approx \sum_{h \in O} X_h^{f32} W_h^{f32} + S^{f32} \cdot \sum_{h \notin O} X_h^{i8} W_h^{i8} \tag{2}$$

其中，X_h^{f32}是输入矩阵X的离群特征维度h的部分，以32位浮点数表示。W_h^{f32}是权重矩阵W的离群值特征维度h的部分，以32位浮点数表示。S^{f32}是用于将8位整数输入和权重矩阵进行反归一化的项。

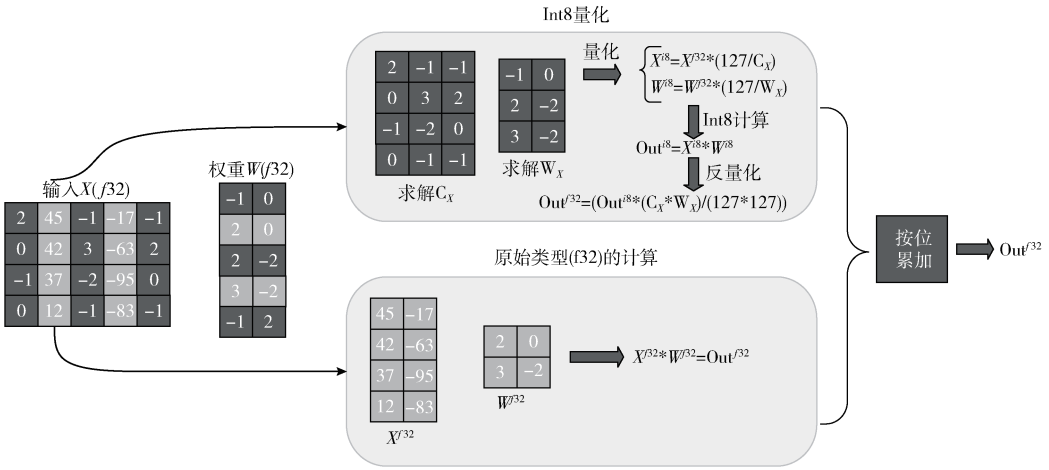

图1　HI8量化

中国知网　https://www.cnki.net

（2）处理非离群值特征维度$h \notin O$的部分

$$\sum_{h \notin O} X_h^{i8} W_h^{i8} \tag{3}$$

其中，X_h^{i8}和W_h^{i8}是输入矩阵X和权重矩阵W的非离群值特征维度h的部分，以8位整数表示。

这种分解方式允许对离群值特征维度进行float16高精度乘法运算，同时使用int8对超过99.9%的值进行内存高效的矩阵乘法。因为对于多达6 B参数的Transformer来说，离群值特征维度的数量不超过6（$|O| \leqslant 6$），因此这种分解操作只额外消耗约0.1%的内存。HI8量化方法有效地平衡了计算精度和内存效率，适用于处理大规模Transformer的矩阵运算。

除了上述混合量化方式，本文针对INT8量化采用推理速率更快的非对称量化策略，即将浮点数值缩放到整数范围内，然后加上一个偏置（零点）来实现。这种方法允许使用整数的所有范围，而不是将整数范围均匀地分配给被量化的数据。具体公式如下。

（1）确定量化范围

参数的最小值min_value和最大值max_value，计算公式如式（4）、式（5）所示

$$\min = \min(\text{参数矩阵}) \tag{4}$$

$$\max = \max(\text{参数矩阵}) \tag{5}$$

（2）计算量化因子

量化范围的最小值q_min和最大值q_max，计算公式如式（6）所示

$$\text{scale_factor} = \frac{\text{q_max}}{\max(|\text{min_value}|, |\text{max_value}|)} \tag{6}$$

（3）计算零点

零点zero_point 参数，计算公式如式（7）所示

$$\text{zero_point} = \text{round}\left(-\frac{\min + \max}{2} \times \text{scale_factor}\right) \tag{7}$$

其中，零点zero_point用于将缩放后的浮点数值偏移，使其落入整数数据类型所需的范围内。零点的设置是通过对量化前的浮点数值进行计算和调整来确定的。

（4）参数量化

浮点数参数P、量化因子scale_factor、零点zero_point，计算公式如式（8）所示

$$X_{i8} = \text{round}(\text{p} \times \text{scale_factor} + \text{zero_point}) \tag{8}$$

对量化结果用$X_{i8} \times \text{scale} + \text{min_value}$进行反量化，将整数值转换回浮点数值，以便输出模型的预测结果。

2.2 HSR方法推理加速

传统KV Cache加速是基于GPU的方法，并且在文本序列较长时，需要消耗大量内存，在缓存不断增加的情况下，每次读取KV时需要消耗较多时间，因此提升效果有限，并不适合CPU的环境[21]。为了解决KV Cache缓存无限制增加的问题，Beltagy提出了滑动窗口技术[23]，但当输入的序列长度大于等于模型支持的最大长度时其模型推理准确率就会大幅下降，甚至会失效。

Nelson等通过大量实验发现大语言模型对序列的起始（primacy bais）和结束（recency bais）部分分配了更多的注意力或权重，而对于中间部分的信息处理能力较弱[24]，所以大语言模型在处理长上下文时，对于信息的位置非常敏感。得益于此结论，本文结合滑动窗口技术提出HSR方法，即主要通过保留最开始的前n个token，以及最近的m个token的Key和Value值，其中最近m个token采用滑动窗口技术来实现。因此HSR方法保留了前n个token的全局语义信息，有助于保持上下文一致性，滑动窗口技术保留最近m个token的局部上下文信息来提高模型生成质量。具体实现如图2所示。

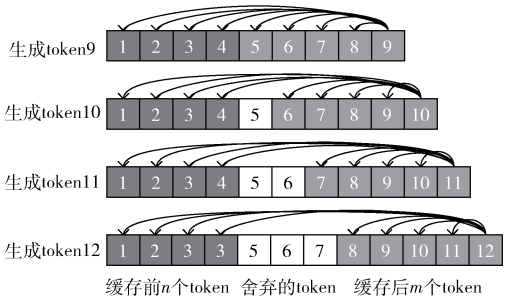

图2 LLMs采用HSR方法加速推理

首先，HSR方法在CPU环境中，维持一个读写Cache，存储生成文本的前n个token的Key值K_{init}，Value值V_{init}，以及最近的m个token的Key值K_{re}，Value值V_{re}，HSR方法流程如下：

（1）初始化阶段：计算输入序列中已输入的token的Key和Value值，保留前n个和此时刻最近的m个Key和Value值对。针对长文本序列的输入，HSR方法会对输入序列进行切片，且只保留第一个切片中的前n个Key和Value值对，在每个切片推理过程中HSR会通过滑动窗口技术来获取此时刻后m个KV值。假设输入tokens为$\{\text{token}_{i1}, \text{token}_{i2}, \cdots, \text{token}_{in}\}$，通过self-attention计算，获取到前$n$个token的$K_{\text{init}}$和$V_{\text{init}}$，以及后$m$个token的$K_{\text{re}}$和$V_{\text{re}}$；

（2）HSR读阶段：类似于传统的KV Cache一样，直接从HSR Cache中获取历史上存储的token的Key和Value值对，即$K_{\text{cache}} = [K_{\text{init}} : K_{\text{re}}]$，$V_{\text{cache}} = [V_{\text{init}} : V_{\text{re}}]$。其中，:符号表示为拼接操作；

（3）HSR更新：假设大语言模型推理执行到第t步，第t步的token的query、key、value分别为q_t、k_t和v_t。将新的数据按式（9）进行更新

$$\begin{aligned} K_{\text{cache}} &= [K_{\text{init}} : K_{\text{re}}[1::] : k_t] \\ V_{\text{cache}} &= [V_{\text{init}} : V_{\text{re}}[1::] : v_t] \end{aligned} \tag{9}$$

中国知网 https://www.cnki.net

其中：符号为拼接，[1::] 为取删除矩阵第一行。

（4）HSR推理：大语言模型推理获取下一个token，最后按照式（10）来计算t步的self-attention输出

$$h_t = \mathrm{softmax}\left(\frac{q_t K_{\mathrm{cache}}^T}{\sqrt{d_k}}\right) V_{\mathrm{cache}} \tag{10}$$

其中，h_t表示的第t步中，输入的token的相对于历史上的token的self-attention输出。大语言模型根据h_t执行后续的步骤，预测得到下个token。

相比于传统的KV Cache方法，HSR在内存管理方面有着显著优势，通过高效控制缓存中Key和Value的存储规模，有效缓解内存占用问题。同时，将HSR与HI8方法相结合，可进一步优化内存使用效率，从而大幅降低缓存所需的额外存储开销。图3展示了在HSR配置中，缓存序列总长度设置为1024（即n+m=1024）条件下，随着输入序列长度增长，保存KV值所需内存大小的变化趋势。

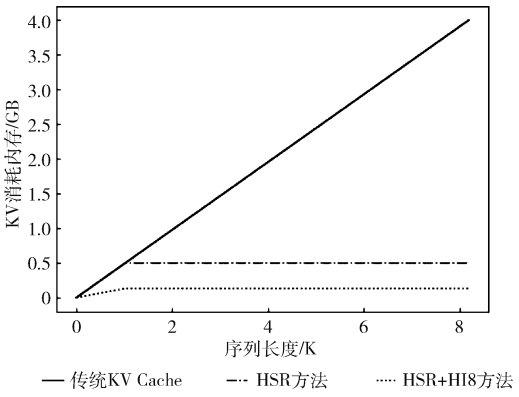

图3 3种KV缓存方式内存消耗情况

从图3结果中可以观察到，传统KV Cache方法的内存需求与输入序列长度呈线性增长关系。当输入序列长度达到8 K时，缓存占用已高达4 GB；随着序列长度扩展至64 K，缓存开销进一步攀升至32 GB。这种内存消耗规模已显著超越大语言模型本身参数加载所需的内存，极大限制了其在实际应用中的可行性。相比之下，HSR方法在输入序列长度小于1 K时，缓存占用同样随序列长度线性增长。然而，当序列长度超过1 K后，HSR通过限制KV存储规模，使缓存内存占用稳定在约0.5 GB，无需随着序列长度的增加而扩展。进一步地，结合HI8量化优化策略后，HSR+HI8方法在相同测试条件下的内存占用进一步降至仅0.13 GB，充分体现两种方法协同优化的效果。

3 实验及结果分析

大语言模型的运行具有跨平台特性，不依赖于特定的操作系统和指令集，其性能主要受限于内存容量、矩阵计算能力以及CPU性能。为确保实验环境的统一性，从而公平地评估不同大模型的性能，本实验选择了消费市场中热度较高且性能可靠的Intel i7-13700 CPU作为计算核心。此外，配置了64 GB内存，以充分满足大语言模型在加载、推理以及上下文缓存存储时对内存的需求。在软件环境上，操作系统选择了Ubuntu LTS 22.04，其作为主流Linux发行版；深度学习框架使用PyTorch 2.1，其广泛支持大语言模型的推理，使模型开发和优化更加便捷；底层库采用OneDNN，旨在利用其在CPU环境下支持矩阵计算。

为全面评估HSR和HI8策略的普适性，本实验选择了3个开源大语言模型进行横向对比，即参数规模为7 B的LLaMA2、6 B的GPT-J和6.7 B的FSEQ。这些模型涵盖了不同的架构与参数规模，其中LLaMA2 7B因其高效的参数设计和广泛的应用场景被学术界广泛研究。

实验采用EleutherAI工具[24]作为统一的评估框架，并选择了3个具有代表性的文本数据集进行性能分析，包括C4[24]、Wikitext[24]和PG19[25]。其中，C4（Common Crawl）涵盖了规模化的网络语料，适合作为大模型通用性能的基础评估集；Wikitext以其结构化内容对模型在上下文约束任务中的表现提供可靠的测试；PG19则专注于长文本生成，适用于评估模型对长序列的记忆能力和上下文一致性。在对不同模型的性能进行对比时，采用困惑度（Perplexity）[25]作为准确度评价指标，采用吞吐量（generated token throughput）[25]作为推理速度评价指标。

3.1 评价指标介绍

3.1.1 Perplexity

Perplexity常用以衡量模型对给定文本的预测能力。其本质是语言模型对测试数据生成概率分布的反向测量，即反映模型在理解或生成语言序列时的“不确定性”或“困惑程度”。具体公式如式（11）所示。

给定序列$X=(x_1, x_2, \cdots, x_N)$，其中$x_i$表示第$i$个词，语言模型的困惑度定义为

$$\mathrm{Perplexity} = 2^{-\frac{1}{N}\sum_{i=1}^{N}\log_2 P(x_i|x_1, x_2, \cdots, x_{i-1})} \tag{11}$$

其中，N是序列长度，$p(x_i|x_1, x_2, \cdots, x_{i-1})$是在先前词$(x_1, x_2, \cdots, x_{i-1})$的条件下预测词$x_i$的概率。

如果模型能够准确预测目标序列中每个词的出现概率，困惑度会较低（接近于1），表示模型性能较好；反之，较高的困惑度表明模型在预测时的表现较差。所以困惑度值越小模型性能越好。

3.1.2 吞吐量

吞吐量是用于衡量语言模型在推理或生成任务中生成文本速度的一个关键性能指标。该指标关注模型每秒生成的token数量（token/s），反映了模型在实际运行中的效率，尤其是在资源有限或需要实时生成的场景下具有重要意义。吞吐量越高，说明模型在计算资源的利用效率上越优。

中国知网 https://www.cnki.net

3.2 HI8量化实验结果与分析

为了全面评估HI8量化的性能，实验基于内存消耗、吞吐量、困惑度3个关键指标对模型实际运行效果进行了分析。此外，实验还引入传统的Int8量化方法和Int8向量式量化（INT8VW）作为对比基线，以验证HI8的性能优势。具体评估包括：对内存消耗和吞吐量的测试基于PG19的8 k输入序列长度，而困惑度采用C4、Wikitext和PG19这3个数据集的平均值作为综合评估标准。实验结果如图4所示。

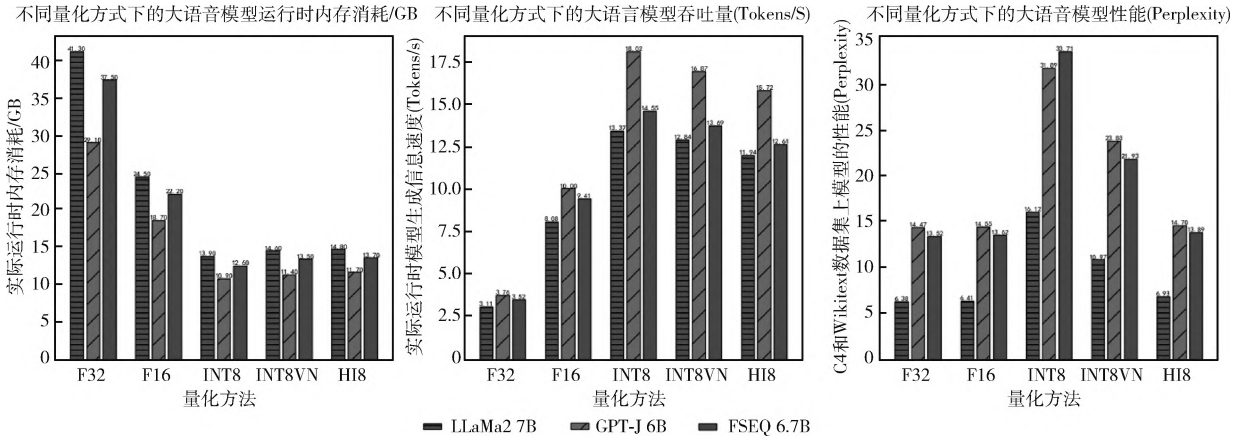

图4 不同量化方法在3个大模型上的性能对比结果

通过观察图4的实验结果不难得出如下结论：

（1）从内存消耗结果可以得出，HI8量化在内存消耗方面表现出显著的优势，与传统的F32精度模型相比，可以实现减少2.67倍的内存。相较于其它量化方法其内存消耗略高，但这种轻微的增加却取得了更优的模型性能；

（2）从模型吞吐量结果可以得出，HI8量化后的模型吞吐量远高于float32的版本，提升了近3.8倍。类似于内存消耗一样，相比于其它传统的量化方法其吞吐量越低，但HI8量化后模型保持着更好的准确性；

（3）从模型困惑度结果可以得出，HI8量化后的模型保持着98%的准确率，而传统的量化方法却只有47%。这验证HI8能够更好地保持模型语义理解能力。

综上，HI8量化方法采用混合精度策略，通过对离群特征使用float16浮点运算，而其余特征则采用Int8非对称量化技术。这种设计在有效压缩模型体积的同时，确保了推理过程中的模型准确率，从而满足大语言模型在CPU环境下本地部署的需求。

3.3 HSR缓存实验结果与分析

HSR是一种针对传统KV Cache在长文本任务中缓存规模过大的问题而提出的优化方法。该方法通过有选择性地保留文本起始和结束部分的关键信息，同时忽略中间部分的冗余信息，在显著降低内存消耗的同时，依然能够有效保留文本的上下文信息，从而保持较高的模型推理准确性。相比传统KV Cache，HSR的主要优势在于以更高的内存效率支持长文本生成任务，而不显著影响模型的性能表现。此外，结合HSR和HI8方法进一步提升了模型在内存管理和推理性能上的表现。结果如图3所示。为验证HSR+HI8方法的有效性，在LLaMA2 7B模型上进行了实验，并在PG19数据集上对模型的困惑度进行了评估。在HSR取最前面8个输入token和最近1012个token的配置下，实验结果如图5所示。

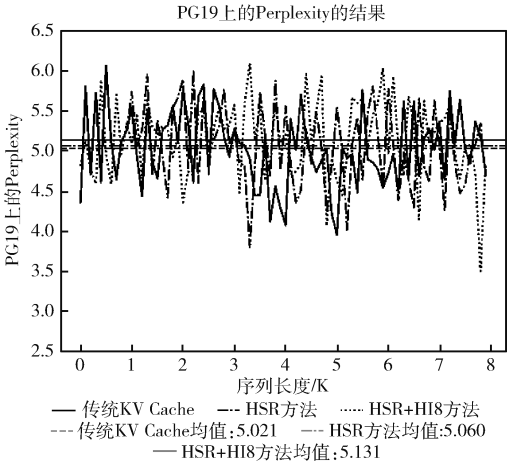

图5 不同KV缓存方法在PG19数据集上的困惑度

结合图3内存消耗和图5的模型准确率的结果中可以看出传统的KV Cache在PG19数据集上的困惑度均值为5.021，表明其具备较高的语言生成能力，但伴随着显著的内存开销。引入HSR后，模型的困惑度均值略微增加至5.06，几乎未对模型的准确性产生影响，同时显著降低了缓存规模。在此基础上，结合HSR和HI8的优化策略，困惑度均值仅小幅提升至5.131，但模型的内存消耗进一步减少，展现出极高的内存管理效率和性能稳定性。因此综合图3和图5的结果可以充分说明HSR+HI8在显著降低内存消耗的同时，仍能保持较高的模型推理准确性。

中国知网 https://www.cnki.net

为进一步分析HI8、HSR以及二者结合（HI8＋HSR）对模型吞吐量的具体影响，实验基于LLaMA2 7B模型，在C4和Wikitext数据集上分别应用这些优化方法，统计各方法在输入序列长度为32、64、128和256条件下的平均吞吐量，因为日常使用中输入序列长度很少超过256个tokens。实验结果如图6所示。

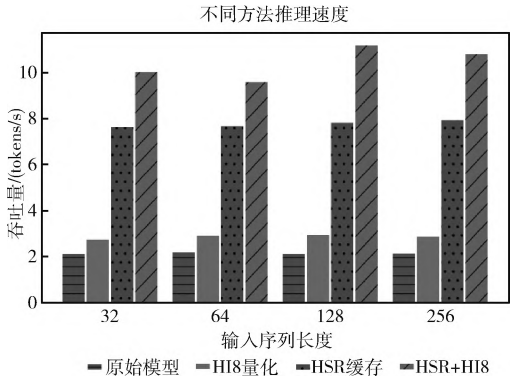

图6　不同方法优化后模型的吞吐量

从图6结果可以看出，使用HI8优化后，模型的吞吐量相较于原始float32提升了约3.6倍，在输入序列长度增加时（32到256），吞吐量保持稳定。HSR方法相比float32略有显著提升，主要是因为输入序列较短，无法充分发挥其优势。HI8与HSR结合在所有输入序列长度下均表现出最优的吞吐量性能，平均提升幅度超过float32的4倍以上。

综上，HI8和HSR有机结合后显著提升了模型的吞吐量，同时保持较高的推理准确性。这种优势使其尤其适合在资源受限的环境中部署大语言模型，满足对高效推理和内存优化的双重需求，从而使得在本地部署和使用大模型成为可能。

3.4　真实案例展示

通过中文和英文提示词测试模型的理解和推理能力，以增强实验的可信性。中文提示词为“谁发明了电话？”，英文提示词为“The cat chased the mouse into the garden，but the mouse escaped by hiding in a ...”。

实验结果如图7和图8所示，其中Message表示用户输入的提示词，Input为整理后的输入日志，第一个Output为原始模型的输出，第二个Output为优化后模型的输出。结果表明，优化后的模型能够更准确地理解用户输入并生成正确的结果。

4　结束语

本文提出HI8和HSR方法，在CPU环境下实现了对大语言模型的显著压缩与加速，成功降低了对高性能GPU的依赖，并显著减少了模型的运行和维护成本。实验评估表明，HI8和HSR在大语言模型上的应用有效降低了内存消耗并提升了推理速度，为大语言模型在CPU环境下的高效应用提供了可行方案。

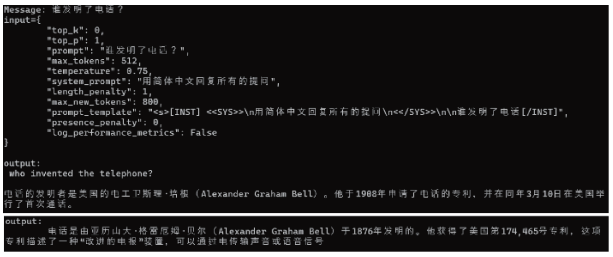

图7　中文提示词模型输出的结果

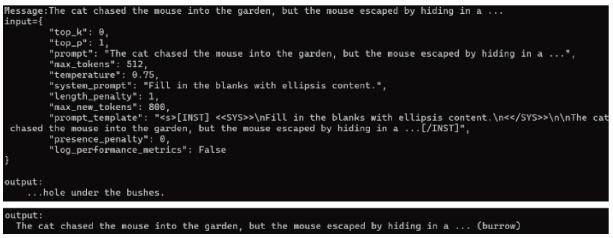

图8　英文提示词模型输出的结果

然而，对于精度要求极高的应用场景，如金融风险评估、医疗诊断等，即使优化后模型的精度损失小于3%，也可能无法满足需求。为此，未来研究需要探索进一步减少量化精度损失的策略，包括但不限于以下方向：改进非对称量化方法，结合量化感知训练微调模型，以及优化中间token丢失策略。

在实际应用中，优化后模型的简便且稳定部署、隐私保护及安全性确保等问题同样重要。为了进一步提升模型部署的稳健性，未来研究引入检查点机制，以强化加载过程中的中断恢复能力。同时，也将积极探索数据加密、访问控制等前沿技术的应用，力求全方位保障用户数据的安全性与隐私性。

参考文献：

[1] Frantar E，Alistarh D. Sparsegpt：Massive language models can be accurately pruned in one-shot［C］//International Conference on Machine Learning，2023：10323-10337.

[2] Wang L，Ma C，Feng X，et al. A survey on large language model based autonomous agents［J］. Frontiers of Computer Science，2024，18(6)：1-26.

[3] Jiang Z，Xu F F，Araki J，et al. How can we know what language models know?［J］. Transactions of the Association for Computational Linguistics，2020，8(2)：423-438.

[4] Sun M，Liu Z，Bair A，et al. A simple and effective pruning approach for large language models［EB/OL］.［2025-02-28］. https://doi.org/10.48550/arXiv.2306.11695.

[5] Carlini N，Tramer F，Wallace E，et al. Extracting training data from large language models［C］//30th USENIX Security Sym-

中国知网　https://www.cnki.net

posium, 2021: 2633-2650.

[6] Jiang S, Fu S, Lin N, et al. Pretrained models and evaluation data for the khmer language[J]. Journal of Tsinghua University, 2022, 27(4): 709-718.

[7] QIAN Yuan. Research and framework implementation of int8 quantization method based on K-L divergence [D]. Beijing: University of Chinese Academy of Sciences, 2023: 1-15 (in Chinese). [钱源. 基于K-L散度的int8量化方法研究与框架实现[D]. 北京: 中国科学院大学, 2023: 1-15.]

[8] Zhao K, Huang S, Pan P, et al. Distribution adaptive INT8 quantization for training CNNs[C]//Proceedings of the AAAI Conference on Artificial Intelligence, 2021: 3483-3491.

[9] Pistellato M, Bergamasco F, Bigaglia G, et al. Quantization-aware NN layers with high-throughput FPGA implementation for edge AI[J]. Sensors, 2023, 23(10): 667-678.

[10] Yang Y, Deng L, Wu S, et al. Training high-performance and large-scale deep neural networks with full 8-bit integers[J]. Neural Networks, 2020, 125(1): 70-82.

[11] Kuzmin A, Van Baalen M, Ren Y, et al. FP8 quantization: The power of the exponent[J]. Advances in Neural Information Processing Systems, 2022, 35(1): 14651-14662.

[12] Sathyaraj P, Devi S R, Kannan K. A Cooperative attack detection framework for IoT network using optimized gradient boosting convolutional neural network[J]. Journal of Circuits, Systems and Computers, 2023, 32(14): 1-27.

[13] Adnan M, Arunkumar A, Jain G, et al. Keyformer: KV cache reduction through key tokens selection for efficient generative inference[J]. Proceedings of Machine Learning and Systems, 2024, 6(1): 114-127.

[14] Ge S, Zhang Y, Liu L, et al. Model tells you what to discard: Adaptive KV cache compression for LLMs[EB/OL]. [2025-02-28]. https://doi.org/10.48550/arXiv.2310.01801.

[15] Wu F, Yang M H, Zhang B, et al. AC-Key: Adaptive caching for LSM-based key-value stores[C]//USENIX Annual Technical Conference, 2020: 603-615.

[16] Brandon W, Mishra M, Nrusimha A, et al. Reducing transformer key-value cache size with cross-layer attention[J]. Advances in Neural Information Processing Systems, 2025, 37(2): 86927-86957.

[17] Zhang Z, Liu S, Chen R, et al. Q-hitter: A better token oracle for efficient llm inference via sparse-quantized KV cache[J]. Proceedings of Machine Learning and Systems, 2024, 6(1): 381-394.

[18] Pope R, Douglas S, Chowdhery A, et al. Efficiently scaling transformer inference[J]. Proceedings of Machine Learning and Systems, 2023, 5(27): 1-14.

[19] Touvron H, Martin L, Stone K, et al. LLaMA 2: Open foundation and fine-tuned chat models [EB/OL]. [2025-02-28]. https://doi.org/10.48550/arXiv.2307.09288.

[20] Gim I, Chen G, Lee S, et al. Prompt cache: Modular attention reuse for low-latency inference [J]. Proceedings of Machine Learning and Systems, 2024, 6(3): 325-338.

[21] ZHOU Yilun. Research on deep learning model quantization and related compression techniques [D]. Shanghai: Shanghai Jiao Tong University, 2020: 1-17 (in Chinese). [周逸伦. 深度学习模型量化及相关压缩技术研究[D]. 上海: 上海交通大学, 2020: 1-17.]

[22] Chu X, Li L, Zhang B. Make RepVGG greater again: A quantization-aware approach [C]//Proceedings of the AAAI Conference on Artificial Intelligence, 2024: 624-632.

[23] Liu T, Hu Y, Gao J, et al. Multi-modal long document classification based on Hierarchical Prompt and Multi-modal Transformer[J]. Neural Networks, 2024, 176(6): 106322.

[24] Liu N F, Lin K, Hewitt J, et al. Lost in the Middle: How language models use long contexts [J]. Transactions of the Association for Computational Linguistics, 2024, 12 (21): 157-173.

[25] Xiao G, Lin, J, Mickael Seznec, et al. SmoothQuant: Accurate and efficient post-training quantization for large language models [J]. International Conference on Machine Learning, 2023, 6(3): 38087-38099.

中国知网 https://www.cnki.net